



STUDENT

0006-ZGS

TENTAMEN

250324 DV2606 2110

Salstentamen (DO)

Kurskod	--
Bedömningsform	--
Starttid	24.03.2025 14:00
Sluttid	24.03.2025 19:00
Bedömningsfrist	--
PDF skapad	05.11.2025 17:15
Skapad av	Anders Göransson

1.1 *SPMD* is a common programming model for writing parallel programs. *SPMD* stands for:

- ☐ Shared-memory Program Many Distributions
- ☐ Shared-memory Programming for Multiple Data
- ☐ None of the alternatives is correct.
- ☒ Single Program Multiple Data
- ☐ Single Processor Multiple threads



Rätt. 1 av 1 poäng.

1.2 The cpu execution time can be described as $T_{exe} = I * CPI * T_c$. Which of the the following alternatives lower the CPI, thus *increasing* the performance?

- ☐ Increase the cache miss latency.
- ☐ Use fewer more complex instructions instead of more simple instructions.
- ☐ None of the alternatives is correct.
- ☒ Increase the instruction-level parallelism.
- ☐ Decrease the cache hit ratio.



Rätt. 1 av 1 poäng.

1.3 Assume a parallel application that is running on a parallel system with a certain efficiency (processor utilization). Which of the following alternatives would *increase* the efficiency?

- ☐ None of the alternatives increases the efficiency.
- ☐ Put more barrier synchronizations in the code.
- ☐ Decrease the input data set size.
- ☒ Increase the input data set size.
- ☐ Increase the number of processors.



Rätt. 1 av 1 poäng.

1.4 Assume a parallel application that is running on a parallel system with a certain efficiency (processor utilization). Which of the following alternatives would *decrease* the efficiency?

- ☐ Decrease the input data set size.
- ☐ Increase the number of processors.
- ☐ Increase the clock frequency of the processors.
- ☐ Put more barrier synchronizations in the code.

☒ All alternatives decrease the efficiency.



Rätt. 1 av 1 poäng.

1.5 What is the (asymptotic) parallel time complexity of *matrix-vector multiplication* using n processors, assuming an $n \times n$ matrix and an $n \times 1$ vector?

☒ $O(n)$



☐ $O(\log n)$

☐ $O(n \log n)$

☐ $O(n^2)$

☐ None of the alternatives is correct.

Rätt. 1 av 1 poäng.

1.6 Simultaneous multithreading is ...

☐ a software approach to parallelize an application

☐ none of the alternatives is correct



☐ a technique to improve branch prediction

☐ a technique to implement multi-level caches

☒ a technique to exploit parallelism in a distributed memory computer



Fel. 0 av 1 poäng.

1.7 Why are snoopy-based cache coherence protocols not used in large-scale shared-memory multiprocessors?

- ☐ They can only be implemented using a write-update policy.
- ☐ Modern processors are too fast for snoopy-based protocols.
- ☒ They are not scalable to a large number of nodes. 
- ☐ They can not guarantee coherence for more than ten caches.
- ☐ None of the alternatives is correct.

Rätt. 1 av 1 poäng.

1.8 MSI (Modified-Shared-Invalid) is a basic write-invalidate protocol. Assuming that a particular cache block is in state Modified, which action is performed when the cache sees a read request on the shared bus?

- ☐ The cache replies with the requested block, and the block stays in state Modified.
- ☐ None of the alternatives is correct.
- ☐ The request is denied since the block is in state Modified.
- ☒ The cache replies with the requested block, and the block is marked as Shared. 
- ☐ The processor is always stalled until the bus request is handled.

Rätt. 1 av 1 poäng.

1.9 In a MIMD parallel computer, the different processors (or processor nodes) are connected through an *interconnection network*. Which of the following metric(s) is(are) used when evaluating the performance of an interconnection network?

☐ Diameter



☐ Bisection bandwidth

☐ Connectivity

☐ Bisection width

☐ All of the mentioned metrics are valid evaluation metrics



Fel. 0 av 1 poäng.

1.10 How many neighbors does each internal node have in a 2-dimensional mesh network with $n \times n$ nodes, i.e., in total n^2 nodes?

☐ $n \log n$

☒ 4



☐ n

☐ n^2

☐ $\log n$

Rätt. 1 av 1 poäng.

2.1 Explain why we need to *protect shared data* in shared memory systems.

Give an example what could happen if we do not protect shared data.

Discuss some common ways to protect shared data in shared memory systems.

In shared memory systems, processes or threads communicate by reading and writing to shared variables or shared data. If this shared data is not protected, i.e. by using barriers, locks, semaphores, this data could be overwritten or the process / thread could retrieve the wrong values, meaning we get incorrect computations and results.

Common ways to protect shared data is via barriers, locks and/or semaphores.

Ord: 65

Besvarad.

2.2 Memory consistency is an important concept for a shared-memory multiprocessor.

Clearly describe what memory consistency means and why it is important. (2p)

Describe two different memory consistency models, including clearly describe how they differ, their advantages and disadvantages, etc. (2p)

Memory consistency, otherwise known as cache coherence, is important to uphold due to when processes / threads have cached copies of shared variables, if a thread modifies its cached copy, other processes / threads would have out-of-date values, meaning incorrect computations and results would occur.

One model for memory consistency in shared-memory is UMA (uniformed memory-access) which has all processors share a single physical memory. For its memory access, the latency and bandwidth is uniform for each processor. Its cache coherence is easy to maintain due to a consistent memory access time for all processors. Although one disadvantage it has is its low scalability, as the memory bus can become a bottleneck for the system.

Another is NUMA (non-uniform memory-access), where each processor uses local memory, although each processor can reach other processors memory with higher latency. For its memory access, the latency is non-uniform, meaning its memory access times are not consistent. This in turn, which becomes its disadvantage, negatively affect cache coherence, as it becomes more difficult to maintain due to the inconsistent memory access times. NUMA has a high scalability, as each processor manages its own memory.

Ord: 189

Besvarad.

2.3 You are working at BTH Embedded System Technologies (BEST), a startup company from BTH. At BEST, you are responsible for developing a new embedded system based on multicore processors and parallel programming. Since the company develops both the hardware and the software, you can decide how many cores that are needed for a particular task. The company management expects the new parallel system to reach a performance speedup of at least 10 times for the program given below as compared to the previous system that was based on a uniprocessor. You are *not allowed to change the program*.

Assume that:

- Each core can complete one floating point operation per clock cycle.
- Cores are clocked at 2 GHz, and each have a 1 MB cache using FIFO replacement.
- The time to service a cache miss (i.e., the time to load the block into the cache) is 200ns.
- All BEST processors (both singlecore and multicore) are attached to a 200 GB/s memory bus.
- Memory latency is perfectly hidden (excellent prefetchers!).

===== start of code =====

```
#define SIZE 1024
```

```
static double A[SIZE][SIZE], B[SIZE][SIZE], C[SIZE][SIZE];
```

```
int main(int argc, char** argv)
```

```
{
```

```
    int i, j, k;
```

```
    double sum, average;
```

```
    // Assume that A and B are initialized before the timer starts (not shown)
```

```
    // Semantics of "parallel_for":
```

```
    // Loop iterations are perfectly partitioned using blocked-cyclic assignment
```

```
    // among P threads each running on one of the processor's P cores.
```

```
    // Timer starts here
```

```
    parallel_for (i = 0; i < SIZE; i++) {
```

```
        for (j = 0; j < SIZE; j++) {
```

```
            c[i][j] = 0.0;
```

```
            for (k = 0; k < SIZE; k++)
```

```
                c[i][j] = c[i][j] + a[i][k] * b[k][j];
```

```
        }
```

```
    }
```

```
    sum = 0.0;
```

```
    for (i = 0; i < SIZE; i++) {
```

```
        for (j = 0; j < SIZE; j++)
```

```
            sum = sum + c[i][j];
```

```
        average = sum / (SIZE * SIZE);
```

```
    // Timer stops here
```

```
}
```

===== end of code =====

a. (3p)

How do you respond to this design challenge? Do you believe you can meet the performance goal?

- If yes, how many cores should be included in the new parallel system?
- If no, explain why.

(justify your answer using e.g. calculations / numbers)

b. (4p)

If you were allowed you to make a few changes to the code, you may be able to deliver better speedup with your multicore processor design. How would you change the code to improve its performance (i.e., improve its speedup)? (describe by using e.g., pseudo code)

- If your answer was NO in part one, how many processors are required to achieve 10 times speedup now?
- If your answer was YES, approximately what speedup do you expect from your previously proposed machine on the new code?

(justify your answer using e.g. calculations / numbers)

speedup of 10 means that: $S(p) = 1 / (\text{par\%} / p + (1 - \text{par\%}))$, and since A and B are already initialized, that leaves the parallel_for and the second for-for-loop. Since we are not allowed to modify the code, the second for-loop (with another loop inside) is a sequential part which is performed for each time, and would become a bottleneck if not adjusted. I do **not** believe that a speedup of 10 is possible with the current code presented.

b) If allowed to modify, I would perform the calculation or summation of values while they are being stored in the c matrix, thereby saving time and efficiently performing the same work without having to iterate through the 1024x1024 matrix two times.

For the processors needed now, I would assume each element in the 1024x1024 c matrix should have its own thread assigned to it to perform each calculation at the same time, and then add it to the sum counter. By having lets say 256 threads per block, 1024 x 1024 would require that many threads, which then could be seen as 2 threads per processor... ---

Ord: 190

Besvarad.

3.1

Select one or more alternatives. Selecting an alternative that is incorrect will give a negative score. The minimum score on this assignment is 0.

- ☐ All the threads in a thread block see the same registers
- ☐ The CPU can access variables in GPU shared memory
- ☒ The content of the CUDA global memory is not changed between two kernel launches in the same program 
- ☒ Threads in different thread blocks cannot access the same variable in GPU shared memory 
- ☐ Accesses to GPU global memory are faster than accesses to GPU shared memory.

Rätt. 2 av 2 poäng.

3.2

Select one or more alternatives. Selecting an alternative that is incorrect will give a negative score. The minimum score on this assignment is 0.

- ☐ The CPU can load values into the constant memory on the GPU 
- ☐ Variables in constant memory can be cached 
- ☐ Constant memory cannot contain floating point numbers
- ☒ Threads in different blocks cannot see the same variables in constant memory 
- ☐ Variables in constant memory are automatically moved to registers

Fel. 0 av 2 poäng.

3.3 Thread Blocks and Warps

Select one or more alternatives. Selecting an alternative that is incorrect will give a negative score. The minimum score on this assignment is 0.

- ☐ Threads in the same Warp can execute on different Streaming Multiprocessors.
- ☒ Threads in the same Warp cannot execute different instructions at the same time. 
- ☐ A Thread Block cannot be larger than a Warp.
- ☒ A Thread Block can contain many Warps. 
- ☐ Threads from different Thread Blocks can execute in the same Warp.

Rätt. 2 av 2 poäng.

3.4 Scatter to Gather

Scatter to gather conversion is a technique that can be used for improving the performance of some CUDA program. Explain how scatter to gather conversion works and why this technique can improve performance. Also give an example of scatter to gather conversion.

Scatter-to-gather works by dividing a large data structure into sub-blocks of data, which are then distributed and stored on the local/cache memory of threads, then these sub-blocks are computed on, and gathered back to the global memory after computation is done. This improves performance since the sub-block of data is stored on the local memory rather than retrieved from the global memory, thereby lowering latency.

An example of scatter-to-gather would be matrix-computations.

Ord: 72

Besvarad.

3.5

Occupancy is defined as the ratio of active warps on an SM to the maximum number of active warps supported by the SM. Why do we normally want to have high occupancy? Which hardware related parameters affect occupancy? Discuss and give examples of how we as programmers, by taking smart design decisions, can provide good trade-offs between the parameters that affect occupancy.

By having high occupancy, we can be assured that many or the warps are being used, in turn covering up the idle process time which occurs during execution.

The parameters which affect the hardware are the grid-size (amount of thread-blocks), block size (amount of threads in each block) and the memory allocated for the process.

The trade-offs we as programmers can do are to make sure our thread blocks sizes are a multiple of 32 (due to warp size being 32).

Ord: 81

Besvarad.

3.6

What is control divergence? Why is control divergence bad for performance? Give an example how we as programmers can avoid or minimize control divergence.

Control divergence means that threads manage to work on the same workload or subdata as other threads, making them perform the same task multiple times or in the same subpart of the data structure. This is bad for performance since it is repeated work and is therefore unnecessary.

We as programmers can avoid this by assigning threads to specific subparts of the data structure, and by having consistent calculations and dynamically adjust the amount of threads or thread blocks used depending on the amount of workload, we can minimize the control divergence.

Ord: 92

Besvarad.

3.7 Tiling

Tiling is a technique that can be used for improving the performance of some CUDA programs, e.g., programs that perform parallel matrix multiplication or convolution. Explain how tiling works, why it can increase performance and why and how we may need to use `__syncthreads()` when using tiling.

Tiling works by dividing the data structure into subsets of data, so called "tiles". Each tile is then loaded by a thread block and stored in its shared memory, then computed on before it is returned and merged back into the global memory. Tiling manages to improve performance by storing each tile in the shared memory, as warps can access the shared memory faster than the global memory, thus improving performance. The main idea of tiling is to improve the performance by using shared memory over global memory, as it is less time costly (compared to hundreds of CPI).

We use `__syncthreads()` to make sure the thread block does not begin computation before it has retrieved all information or before it has written back the values calculated, as warps do not always finish at the same time. We can use `__syncthreads()` after loading in a tile, to ensure all data is loaded before computation, and after we finish computation and before loading the next one in, making sure each warp has finished and that its calculated value is not overwritten.

Ord: 180

Besvarad.

3.8

Explain how sparse matrices are stored when using the ELL format. Also explain the potential performance advantages for a CUDA program when using the ELL format compared to the CSR (compressed sparse row) format. Also, explain how sparse matrices are stored when using the COO (coordinate list) format. Also, explain why we sometimes would like to combine the ELL and COO formats when using CUDA. Give an example of how the ELL and COO formats can be combined when storing a sparse matrix.

The COO stores coordinates by using three arrays: one for row, one for column, and last for value. By doing this, we avoid having to store zero-values.

As an example, by having a 2x2 matrix with the arrays row = [0,1], column = [0,1] and values = [6, 12] we get the matrix:

6	0
0	12

Ord: 57

Besvarad.

3.9 Thread coarsening

Select one or more alternatives. Selecting an alternative that is incorrect will give a negative score. The minimum score on this assignment is 0.

- ☒ Thread coarsening can reduce the number redundant calculations in a CUDA program 
- ☐ Thread coarsening makes it possible to perform data transfer between the CPU and GPU in parallel with kernel execution on the GPU
- ☐ Thread coarsening increases the parallelism in a CUDA program
- ☐ Thread coarsening decreases the completion time of a thread
- ☒ Thread coarsening can make it possible to use the shared memory more efficiently in a CUDA program 

Rätt. 2 av 2 poäng.